FixFlow Integration: End-to-End User Flow

Let's say you have an existing application built with FastAPI, Node.js, Django, or another backend framework. You want to integrate **FixFlow**, an automated incident investigation and code-fixing system, into your application.

The goal is:

Your application encounters a problem → FixFlow detects it → investigates the root cause → generates a fix → tests the fix → creates a GitHub Pull Request → notifies your team.

Based on your architecture diagram, FixFlow has three major responsibilities:

1. **Monitoring and incident detection:** Detect problems using metrics, logs, and traces.
2. **Automated investigation and remediation:** Understand the problem, identify the faulty code, and propose a fix.
3. **Validation and delivery:** Run tests, verify the fix, and create a Pull Request for review.

The important architectural decision is that **FixFlow should integrate with your existing application and development workflow rather than replace your application or its monitoring stack.**

Let's walk through the integration from the perspective of a developer who wants to onboard an existing application.

---

1. Understand the Overall Architecture

Suppose you have an application called `PaymentService`.

It has:

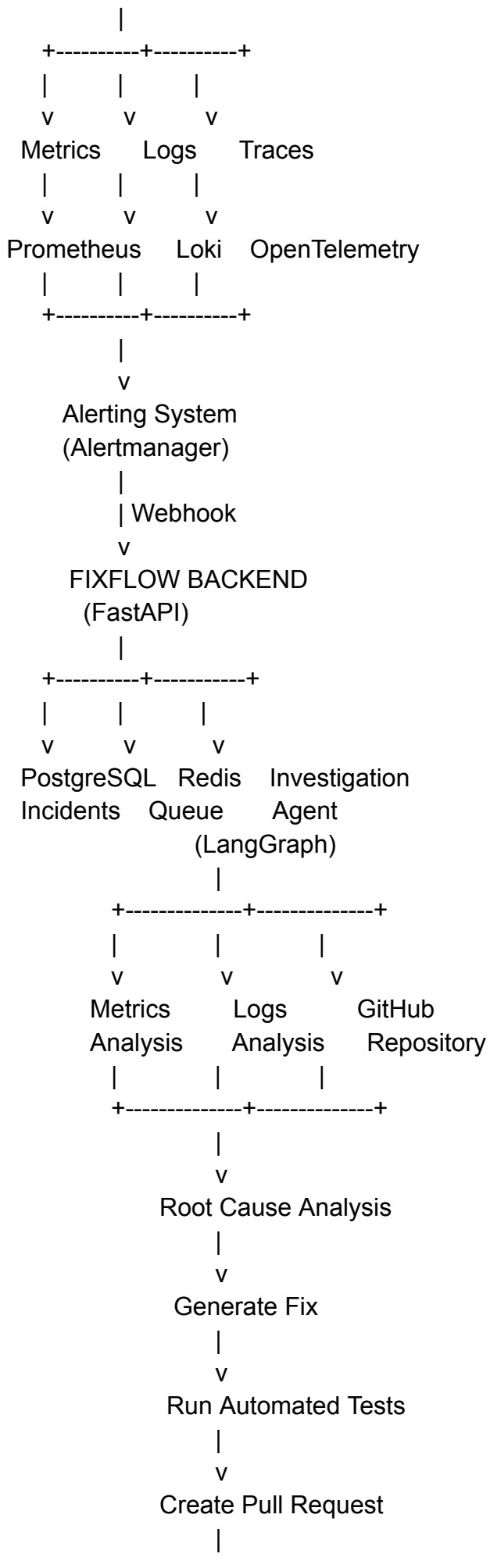
- A FastAPI backend.
- A PostgreSQL database.
- A GitHub repository.
- Existing production users.
- Prometheus and Loki for monitoring.

You want FixFlow to automatically investigate incidents such as payment failures, slow API responses, and application crashes.

The architecture would look like this:

text

```
YOUR APPLICATION
 PaymentService
```


Metrics
Logs
Traces
Prometheus
Loki
OpenTelemetry
Alerting System
(Alertmanager)
Webhook
FIXFLOW BACKEND
(FastAPI)
PostgreSQL
Incidents
Redis
Queue
Investigation
Agent
(LangGraph)
Metrics
Analysis
Logs
Analysis
GitHub
Repository
Root Cause Analysis
Generate Fix
Run Automated Tests
Create Pull Request

v
Notify Stakeholders

Here, the monitoring stack observes your application, while FixFlow consumes the resulting incident information and performs the investigation.

You do not need to build another monitoring system if your application already has one. You need to connect it to FixFlow.

2. User Flow: How a Developer Integrates FixFlow

From your perspective as a developer, the integration can be divided into **six phases**.

text
1. REGISTER APPLICATION
|
v
2. CONNECT GITHUB REPOSITORY
|
v
3. CONNECT MONITORING STACK
|
v
4. CONFIGURE ALERTS & WEBHOOK
|
v
5. CONFIGURE FIXFLOW AGENT
|
v
6. TEST INTEGRATION
|
v
FIXFLOW IS READY

Let's go through each phase and understand what the developer actually does.

Phase 1: Register Your Application with FixFlow

Suppose you have an application named `PaymentService`.

You open the FixFlow dashboard and click **Add Application**.

You provide the following details:

Field	Example

Application name	PaymentService
Environment	Production
Application URL	`https://payments.example.com`
Backend framework	FastAPI
Programming language	Python
Repository	`company/payment-service`
Monitoring system	Prometheus + Loki
Deployment method	Docker

FixFlow creates an application record and assigns it an identifier.

For example:

```
json
{
  "application_id": "app_001",
  "name": "PaymentService",
  "environment": "production",
  "framework": "fastapi",
  "language": "python",
  "repository": "company/payment-service",
  "monitoring": {
    "metrics": "prometheus",
    "logs": "loki"
  }
}
```

This is illustrative configuration, not an existing FixFlow API contract.

What happens behind the scenes?

FixFlow stores the application configuration in PostgreSQL.

It can associate future incidents with this application and determine which repository, monitoring sources, and environment should be used during an investigation.

Developer's responsibility: Register the application and provide its configuration.

FixFlow's responsibility: Store the application details and prepare its integration configuration.

Phase 2: Connect Your GitHub Repository

Next, FixFlow needs access to your application's source code.

Why?

Because collecting metrics and logs can reveal that something is wrong, but investigating the code helps identify where the problem originates.

For example, your repository contains:

text
```
payment-service/
├── app/
│   ├── main.py
│   ├── routes/
│   │   └── payments.py
│   ├── services/
│   │   └── payment_service.py
│   └── database.py
├── tests/
├── requirements.txt
└── Dockerfile
```

FixFlow needs to be able to inspect the repository to:

- Read relevant source files.
- Identify the code associated with an incident.
- Create an isolated working branch.
- Propose a patch.
- Run tests against the changes.
- Create a Pull Request.

What does the developer do?

1. Click **Connect GitHub**.
2. Authorize FixFlow using a GitHub App or another supported authentication mechanism.
3. Select the repository.
4. Configure the default branch, such as `main`.
5. Grant the minimum permissions required for the workflow.

For example:

text
```
Repository: company/payment-service

Default branch: main
```

Permissions:
Read source code: Yes
Create branches: Yes
Read workflow results: Yes, if needed
Create pull requests: Yes
Merge pull requests: No, by default

The precise permissions depend on the features FixFlow implements.

Important: FixFlow should not need permission to merge into production automatically. Start with creating Pull Requests so that a developer can review and approve the changes.

Phase 3: Connect Prometheus, Loki, and Tracing

This is where your earlier question about Grafana, Prometheus, and Loki becomes relevant.

FixFlow needs evidence to understand what happened during an incident.

You configure the monitoring data sources that FixFlow can query.

A. Connect Prometheus

Provide the Prometheus endpoint.

For example:

text
http://prometheus:9090

FixFlow uses PromQL queries to retrieve metrics such as:

- CPU utilization.
- Memory usage.
- HTTP request rate.
- HTTP error rate.
- Request latency.

Example query:

promql
sum(rate(http_requests_total{status=~"5.."}[5m]))

This calculates the rate of HTTP requests with status codes in the 500 range, assuming your application exposes the relevant metric.

B. Connect Loki

Provide the Loki query endpoint.

For example:

text
http://loki:3100

FixFlow uses LogQL to search application logs.

Example:

logql
{service_name="payment-service"} |= "ERROR"

This searches for log entries containing `ERROR` for the specified service, assuming your logs use that label.

C. Connect OpenTelemetry

If you collect distributed traces, FixFlow can use them to understand the path of a request across different services.

For example:

text
API Gateway
    |
    v
Payment API
    |
    v
Payment Service
    |
    v
PostgreSQL

A trace may reveal that a request spent most of its time waiting for PostgreSQL.

That information helps narrow down the investigation.

What does the developer configure?

Configuration	Purpose
Prometheus endpoint	Retrieve metrics

Loki endpoint	Retrieve logs
Trace query endpoint	Retrieve distributed traces, if available
Authentication credentials	Authorize queries
Environment and service labels	Identify the relevant application's data
Retention and query limits	Determine which evidence is available

You do not necessarily need all three data sources on day one. You can start with metrics and logs, then add tracing later.

Key point: Grafana is optional for the data retrieval path if FixFlow can query Prometheus, Loki, and your tracing backend directly through their APIs. Grafana can still provide dashboards and links that developers use to inspect the incident.

Phase 4: Configure Alerting and the Webhook

This is the most important part of the integration.

How does FixFlow know that an incident has occurred?

You need an event source that tells FixFlow when something requires investigation.

In the architecture you shared, that source is Alertmanager.

The flow is:

text
```
PaymentService
     |
     v
 Prometheus
     |
     | Detects elevated error rate
     v
 Alertmanager
     |
     | HTTP POST webhook
     v
 FixFlow Backend
     |
     v
 Create Incident
```

Prometheus evaluates metrics and generates alerts. Alertmanager handles alert grouping, deduplication, and routing to receivers.

You can configure a webhook receiver to send alerts to FixFlow. See the official Prometheus Alertmanager documentation.

Step 4.1: Create an alert rule

Suppose your application normally has a low error rate.

You want an incident to be raised when the 5xx error rate exceeds 10 requests per second for two minutes.

An illustrative Prometheus alert rule is:

```
yaml
groups:
  - name: payment-service-alerts
    rules:
      - alert: HighPaymentErrorRate
        expr: |
          sum(rate(http_requests_total{
            service="payment-service",
            status=~"5.."
          }[5m])) > 10
        for: 2m
        labels:
          severity: critical
          service: payment-service
          environment: production
        annotations:
          summary: "High error rate in PaymentService"
          description: "HTTP 5xx errors have exceeded 10 requests per second."
```

This rule assumes the application exposes `http_requests_total` with the specified labels. You would adapt the query to your actual metrics.

Step 4.2: Configure the webhook

FixFlow exposes an endpoint such as:

```
text
POST /api/v1/webhooks/alertmanager
```

This is an example endpoint that you would implement in FixFlow.

In Alertmanager, configure the receiver to point to it:

yaml

```
receivers:
  - name: fixflow
    webhook_configs:
      - url: "https://fixflow.example.com/api/v1/webhooks/alertmanager"
        send_resolved: true
```

The full Alertmanager configuration also needs an appropriate routing rule that directs the relevant alerts to the `fixflow` receiver.

When an alert fires, Alertmanager sends an HTTP request containing alert information.

Alternatively, if you're using Grafana-managed alerting, you can configure a webhook contact point in Grafana's Alerting interface. See the official Grafana webhook configuration guide.

You generally choose the alerting route that matches your existing setup; you don't need to configure both just to send the same alert to FixFlow.

Step 4.3: What does FixFlow receive?

A simplified alert payload might look like this:

```
json
{
  "status": "firing",
  "alerts": [
    {
      "status": "firing",
      "labels": {
        "alertname": "HighPaymentErrorRate",
        "service": "payment-service",
        "environment": "production",
        "severity": "critical"
      },
      "annotations": {
        "summary": "High error rate in PaymentService"
      }
    }
  ]
}
```

The actual payload includes additional fields and depends on the alerting system.

FixFlow's webhook handler should then:

1. Authenticate the incoming request.
2. Validate the payload.
3. Identify the application using its configured service or environment labels.
4. Check whether the event represents a firing or resolved alert.

5. Deduplicate repeated notifications.
6. Create or update the incident in PostgreSQL.
7. Enqueue an investigation job in Redis.

For example:

text
Webhook received
|
v
Validate request
|
v
Identify application
|
v
Check duplicate incident
|
v
Create incident in PostgreSQL
|
v
Enqueue job in Redis
|
v
Return HTTP 2xx

Return a successful response after the event has been accepted and durably recorded or queued. Do not wait for the entire AI investigation to finish before acknowledging the webhook.

Also, configure authentication and request verification, use HTTPS, and make webhook processing idempotent because notification systems may retry deliveries.

Phase 5: Configure the FixFlow Investigation Agent

Now that FixFlow can receive incidents and access monitoring data, you need to configure its investigation workflow.

In your architecture, this is handled by the LangGraph agent.

The agent should receive an incident context such as:

json
```
{
  "incident_id": "inc_001",
  "application_id": "app_001",
  "service": "payment-service",
```

```
  "environment": "production",
  "alert": "HighPaymentErrorRate",
  "severity": "critical",
  "repository": "company/payment-service",
  "status": "RECEIVED"
}
```

The agent can then execute a stateful investigation workflow.

Investigation flow

```
text
Incident received
      |
      v
Collect evidence
      |
      +----> Query Prometheus
      |
      +----> Query Loki
      |
      +----> Query tracing backend
      |
      +----> Retrieve deployment information
      |
      v
Analyze evidence
      |
      v
Identify suspected root cause
      |
      v
Inspect relevant source code
      |
      v
Reproduce the problem in isolation
      |
      v
Generate candidate fix
```

For example, FixFlow discovers these events:

Prometheus:

```
text
Payment API error rate increased at 10:15.
```

Loki:

text
Database connection timeout.

Deployment information:

text
A new application version was deployed at 10:14.

GitHub:

text
Recent changes modified the database connection pool.

The agent combines these signals to form a root-cause hypothesis.

It should distinguish a confirmed root cause from a suspected one and record the evidence supporting its conclusion.

It then inspects the relevant code and attempts to reproduce the failure in an isolated environment.

Developer's responsibility: Configure the repository, data-source access, execution environment, and policies.

FixFlow's responsibility: Gather evidence, correlate the available information, investigate the code, and propose a fix.

Phase 6: Configure Testing and Pull Request Creation

Finding a plausible cause does not mean the fix is correct.

FixFlow should validate its proposed changes before presenting them to your team.

Step 6.1: Generate a patch

Suppose the agent identifies an incorrectly configured database connection timeout.

It creates a working branch:

text
fixflow/inc-001-db-timeout

It then applies the proposed code change to that branch.

Step 6.2: Run tests in isolation

The agent launches an isolated test environment, such as a Docker-based environment.

It executes:

text
1. Unit tests
2. Integration tests
3. Regression tests
4. Optional performance tests

The exact tests depend on your application's test suite and the incident.

The agent records:

- Which tests passed.
- Which tests failed.
- Whether the original issue was reproduced.
- Whether the proposed fix addresses the failure.
- Whether the change introduces regressions.

Do not run untrusted AI-generated code directly in your production environment. Use a sandbox with restricted permissions, controlled network access, and limited access to secrets.

Step 6.3: Create a Pull Request

If the patch meets your validation criteria, FixFlow creates a Pull Request in your repository.

The PR might contain:

text
Title:
Fix database connection timeout in PaymentService

Description:
Incident: inc_001

Problem:
Payment requests were failing because of database
connection timeouts.

Suspected root cause:
The database connection configuration did not handle
the observed workload correctly.

Changes:
- Updated connection timeout configuration.
- Added a regression test.

Validation:
- Unit tests: Passed
- Integration tests: Passed
- Performance tests: Not run

Review:
Human review required before merging.

This is an illustrative PR description.

The Pull Request gives your team a reviewable record of the incident, the proposed fix, and the test results.

Step 6.4: Notify stakeholders

Finally, FixFlow sends a notification through your configured email or messaging integration.

For example:

text
FixFlow Incident Report

Application: PaymentService
Incident: inc_001

Problem:
High HTTP 5xx error rate.

Investigation:
Database connection timeouts were observed.

Proposed fix:
Updated database connection handling.

Validation:
Unit and integration tests passed.

Pull Request:
https://github.com/company/payment-service/pull/123

Next step:
Review and approve the proposed changes.

The example GitHub link is illustrative.

At this point, the integration has completed its primary workflow: detection, investigation, patch generation, validation, and PR creation.

---

3. What Does the Developer Actually Need to Change in Their Application?

This is an important distinction: **integrating FixFlow should not necessarily require modifying your application's business logic.**

The required changes depend on what your application already exposes.

Requirement	If already available	If not available
Metrics	Connect your existing metrics endpoint	Instrument the application or deploy a suitable exporter
Logs	Configure your existing log collector	Set up structured logging and a log collection pipeline
Distributed traces	Connect your existing tracing backend	Add OpenTelemetry instrumentation and configure trace collection
Error detection	Reuse existing alert rules	Create alert rules for important failure conditions
Source code	Connect the GitHub repository	Set up repository access
Automated testing	Reuse your test suite	Add suitable unit and integration tests
Reproduction environment	Reuse a suitable staging or Docker setup	Create an isolated environment for reproducing incidents

For example, suppose you already have FastAPI, Prometheus, Loki, and GitHub configured.

You may only need to:

1. Register your application in FixFlow.
2. Connect the existing monitoring endpoints.
3. Connect your GitHub repository.
4. Configure the alert webhook.
5. Set up the test environment and investigation policies.

You don't need to replace your monitoring stack or rewrite your application.

---

4. What Should the FixFlow Onboarding UI Look Like?

If you're building FixFlow itself, I'd recommend implementing onboarding as a guided setup wizard.

Screen 1: Add Application

The developer enters:

- Application name.
- Framework and programming language.
- Environment.
- Repository URL.

Button: **Next**

Screen 2: Connect Source Control

The developer authorizes GitHub and selects the repository.

FixFlow verifies that it can read the source code and create a test branch or Pull Request, according to the granted permissions.

Status example:

text
GitHub Connection

Repository: company/payment-service

✓ Repository accessible
✓ Default branch detected
✓ Pull Request permission verified

Screen 3: Connect Observability

The developer configures the available data sources.

text
Observability Setup

Prometheus
[Endpoint URL]
[Test Connection]

Loki
[Endpoint URL]
[Test Connection]

Tracing
[Endpoint URL]
[Test Connection]

Each connection should be tested independently.

Screen 4: Configure Incident Detection

FixFlow explains how to connect the alerting system.

text
Incident Detection

Webhook endpoint:

https://fixflow.example.com/api/v1/webhooks/alertmanager

[Copy Endpoint]

Authentication:
[Configure credentials]

[Test Webhook]

FixFlow should provide setup instructions appropriate to the selected alerting system.

Screen 5: Configure Investigation Policies

The developer chooses what FixFlow is permitted to do.

For example:

Setting	Recommended initial value
Automatic investigation	Enabled
Automatic patch generation	Enabled
Run tests in isolation	Enabled
Create Pull Requests	Enabled
Merge Pull Requests automatically	Disabled

Deploy fixes automatically	Disabled

These settings should be enforced by the backend, not merely hidden or disabled in the UI.

Screen 6: Verify Integration

FixFlow performs a readiness check.

text
Integration Health

✓ Application registered
✓ GitHub connected
✓ Prometheus reachable
✓ Loki reachable
✓ Alert webhook verified
✓ Investigation worker available
✓ Test environment configured

Status: READY

The tracing check should only be required if tracing is enabled for that application.

After setup, the developer can navigate to the application's incident dashboard.

---

5. What Happens After Integration Is Complete?

From the developer's perspective, FixFlow should operate in the background.

The typical workflow becomes:

text
Developer deploys application
        |
        v
Monitoring stack observes application
        |
        v
An alert fires
        |
        v
FixFlow creates an incident
        |
        v
FixFlow gathers metrics, logs, and traces

|
v
AI investigates the root cause
|
v
AI generates a patch
|
v
FixFlow runs tests
|
v
FixFlow creates a Pull Request
|
v
Developer reviews the changes
|
v
Developer approves and merges
|
v
Normal deployment pipeline deploys the fix

The final deployment should go through your existing CI/CD process unless you explicitly design and authorize a separate automated remediation mechanism.

FixFlow does not need to sit in the application's request-response path. The application can continue serving requests while FixFlow investigates incidents asynchronously.

6. The Most Important Architectural Decision

I would separate FixFlow into two parts.

Part A: One-time application integration

This handles:

- Application registration.
- GitHub authorization.
- Monitoring configuration.
- Webhook setup.
- Testing and security policies.

The developer performs these steps during onboarding and revisits them when configuration changes.

Part B: Automatic incident processing

This handles:

- Receiving alerts.
- Creating incidents.
- Collecting evidence.
- Investigating root causes.
- Generating patches.
- Running tests.
- Creating Pull Requests.
- Sending notifications.

This runs automatically whenever a qualifying incident occurs.

The key design principle is that onboarding should be a one-time process, while incident investigation should be an event-driven background workflow.

One final point: your diagram represents the intended FixFlow architecture, not a verified implementation with existing endpoints or supported integrations. The API paths, configuration fields, and onboarding screens above are proposed examples that you can use when designing the actual system.